
Especificación ejecutable, modelado y momento de la arquitectura

Nivel [N1], [N2] y [N3] – familias SPC, MOD y ARQ

Documento **SLCP-DOC-008** – Versión **1.1**

6 de agosto de 2026

Este documento resuelve tres cuestiones planteadas de forma expresa: qué formato de especificación ejecutable conviene generar por cada requisito, qué diagramas deben derivarse y cómo se conserva la trazabilidad desde el requisito hasta el último de ellos, y en qué momento del ciclo debe definirse la arquitectura del sistema junto con su vista de despliegue. Señala además una colisión terminológica que debe resolverse antes de implementar el modelado, porque el término componente designa hoy dos conceptos distintos dentro del proyecto.

Cambios de la versión 1.1: MOD-01 se concilia con la edición gráfica introducida en PRO-01 de SLCP-DOC-009, y se cierra Q-31.

Índice

1 Colisión terminológica que debe resolverse primero	2
2 Especificación ejecutable por requisito	2
2.1 Análisis del formato	2
2.2 Recomendación: dos capas	3
2.3 Por qué esto encaja con lo ya decidido	3
2.4 Requisitos	3
3 Modelado y diagramas	4
3.1 El diagrama es una vista del modelo, no un documento	4
3.2 Diagramas por perspectiva	5
4 Momento de definir la arquitectura	6
4.1 Análisis	6
4.2 Una consecuencia que conviene no pasar por alto	6
4.3 Requisitos	6
4.4 Vista de despliegue	7
5 Vocabulario añadido	7
6 Asuntos abiertos	8

1 Colisión terminológica que debe resolverse primero

El término *componente* designa en este proyecto dos conceptos que no tienen relación entre sí.

Concepto	Origen	Naturaleza
Componente de la estructura de desglose	SLCP-DOC-005, nivel intermedio entre entregable y tarea	Unidad de <i>trabajo</i> : algo que alguien hace
Componente de software	Modelado UML y arquitectura	Unidad de <i>construcción</i> : algo que el sistema tiene

NAM-03 prohíbe que un identificador designe dos conceptos del vocabulario cerrado, de modo que uno de los dos debe cambiar. Se propone conservar `COMPONENT` como literal del nivel de desglose, por ser el término que emplea la parte interesada, y denominar `SoftwareComponent` al elemento arquitectónico.

La distinción tiene consecuencia práctica inmediata: cuando el enunciado pide un diagrama de casos de uso detallado por componente, se refiere al componente de trabajo, porque es la unidad en que se organiza el proyecto. El diagrama de componentes de software es un artefacto distinto, propio de la vista estática de la arquitectura, y no se corresponde uno a uno con aquél.

2 Especificación ejecutable por requisito

2.1 Análisis del formato

Se solicitó analizar qué formato es hoy el más adecuado. La conclusión es que ninguno cubre por sí solo el abanico de requisitos que un sistema de gestión contiene, y que la elección debe hacerse por tipo de requisito y no por preferencia global.

Tabla 1: Formatos considerados

Formato	Fortaleza	Límite
Gherkin, con la estructura dado-cuando-entonces	Legible por quien no programa, ejecutable por herramienta, y traducible a caso de prueba sin pérdida	Degenera ante lógica combinatoria: una regla con seis condiciones produce decenas de filas de ejemplo. No expresa invariantes
Sintaxis EARS [1]	Restringe el enunciado del requisito a cinco patrones, lo que ataca de raíz la ambigüedad y la vaguedad	No describe escenarios ni sirve de oráculo por sí sola
Tabla de decisión DMN	Expresa la lógica combinatoria una sola vez y genera de ella tanto el código como los casos de prueba	Ilegible para escenarios narrativos; no captura secuencia temporal
Máquina de estados	Captura el comportamiento dependiente de la historia y permite derivar cobertura de transiciones	No expresa cálculo ni reglas de decisión

Formato	Fortaleza	Límite
Propiedades e invariantes	Expresan lo que debe cumplirse siempre, no solo en los casos enumerados	Requieren formación que no puede presuponerse en la parte interesada

2.2 Recomendación: dos capas

Capa de enunciado: EARS. El requisito se escribe conforme a los patrones de Mavin et al. [1], que fijan un orden de cláusulas y un vocabulario reducido de palabras clave. La ventaja concreta para esta plataforma es que convierte la comprobación de ANA-01 en algo tratable: verificar la conformidad con una plantilla es muchísimo más fiable que emitir un juicio sobre texto libre, y permite señalar con precisión qué falta.

Capa de comportamiento: Gherkin como formato principal, con tres complementos. El escenario dado-cuando-entonces se mantiene como formato por defecto porque es el único que el propietario del producto puede leer y aprobar sin intermediario, y esa propiedad es indispensable aquí: ROL-03 le atribuye la aceptación, y una aceptación sobre artefactos que no entiende es una firma vacía. Los complementos se aplican cuando el escenario degenera: tabla de decisión para la lógica combinatoria, máquina de estados para el comportamiento dependiente de la historia, e invariantes para lo que debe cumplirse siempre.

2.3 Por qué esto encaja con lo ya decidido

El escenario Gherkin no es un artefacto más: es simultáneamente el criterio de aceptación que TRC-06 exige, el oráculo sellado que TGT-08 impone antes de generar implementación, y el caso de prueba que TRC-12 reclama. Un solo artefacto cumple tres obligaciones que hoy figuran por separado, y lo hace en una forma que el propietario del producto puede aprobar.

Conviene tener presente una cautela empírica. El trabajo reciente sobre generación automática de escenarios muestra validez estructural muy alta cuando la generación se restringe con un esquema, pero también que el no determinismo persiste entre ejecuciones y que las garantías deben revalidarse ante cada cambio de versión del modelo generador [2]. Es exactamente lo que TRC-23 y ANA-05 ya exigen declarar, y lo que R-03 de SLCP-ADR-0002 identificó como deriva del motor.

2.4 Requisitos

SPC-01 Enunciado conforme a EARS. [N1][N2]

La plataforma debe ofrecer los patrones de Mavin et al. [1] para la redacción de requisitos y el análisis de ANA-01 debe evaluar la conformidad con ellos. El uso de los patrones debe ser recomendado y no obligatorio.

Verificación: El análisis identifica el patrón empleado o su ausencia, y ningún requisito se rechaza por no seguirlo.

SPC-02 Escenarios por requisito. [N1][N2][N3]

Por cada requisito aprobado la plataforma debe generar escenarios en formato Gherkin que

cubran el caso nominal, los casos límite y los casos de error derivados de sus criterios de aceptación.

Verificación: Ningún requisito aprobado carece de al menos un escenario, y cada escenario referencia el requisito que lo origina.

SPC-03 Selección del formato complementario. [N1][N2]

Cuando el requisito exprese lógica combinatoria, comportamiento dependiente de estado o una invariante, la plataforma debe proponer respectivamente una tabla de decisión, una máquina de estados o una propiedad, en lugar de escenarios enumerados.

Verificación: Un requisito con más de un umbral configurable de condiciones combinadas no produce una lista de escenarios sino una tabla de decisión.

SPC-04 El escenario es el oráculo. [N1][N2][N3]

Los escenarios aprobados constituyen el oráculo sellado que exige TGT-08. Su modificación posterior queda sujeta a la aprobación independiente que ese requisito establece.

Verificación: No existen dos artefactos distintos para criterio de aceptación y oráculo: son el mismo, con una sola versión vigente.

SPC-05 Aprobación por el propietario. [N1][N2]

Los escenarios deben someterse a la aprobación del propietario del producto junto con el requisito que los origina, por ser la forma concreta en que ese rol declara qué significa que el requisito esté satisfecho.

Verificación: Ningún escenario alcanza estado de oráculo sin aprobación del propietario registrada.

3 Modelado y diagramas

3.1 El diagrama es una vista del modelo, no un documento

MOD-01 Diagramas derivados del modelo. [N1][N2]

Todo diagrama debe generarse como proyección del modelo del proyecto y no debe existir como artefacto independiente. El diagrama es editable conforme a PRO-01, y su edición constituye una edición del modelo: la superficie gráfica es un modo de editar el modelo, no un lienzo aparte. Únicamente la disposición visual se persiste fuera del modelo, como preferencia de presentación.

Verificación: Regenerar un diagrama a partir del modelo reproduce exactamente el resultado de cualquier edición realizada sobre él.

Este requisito resuelve por construcción lo que el enunciado pide conservar. Si cada diagrama es una proyección del mismo grafo, la trazabilidad desde el requisito hasta el último diagrama no se mantiene: es inherente, porque no hay nada separado que pueda desincronizarse. La alternativa, diagramas como documentos que alguien actualiza, es la que produce la situación conocida de un modelo que dice una cosa y un diagrama que dice otra.

3.2 Diagramas por perspectiva

Se recogen las tres perspectivas clásicas del análisis — datos, funciones y comportamiento — y se añaden las dos vistas arquitectónicas.

Tabla 2: Diagramas y su origen en el modelo

Perspectiva	Diagrama	De qué se deriva
Funciones	Casos de uso de grano grueso, por proyecto	Requisitos aprobados agrupados por entregable
Funciones	Casos de uso detallados, por componente de trabajo	Requisitos que contribuyen a ese componente vía <code>contributesTo</code>
Funciones	Diagrama de actividad	Flujo de los escenarios de un caso de uso
Funciones	Proceso de negocio en BPMN [3]	Procesos que atraviesan varios casos de uso
Datos	Diagrama de clases del dominio [4]	Glosario y entidades del modelo de dominio
Datos	Modelo relacional y esquema	Diagrama de clases, vía las reglas de correspondencia y NAM
Comportamiento	Máquina de estados	Entidades con ciclo de vida declarado
Comportamiento	Diagrama de secuencia	Un diagrama por escenario Gherkin
Decisión	Tabla de decisión DMN [5]	Reglas de negocio combinatorias
Arquitectura	Diagrama de componentes de software	Elementos arquitectónicos y sus interfaces
Arquitectura	Diagrama de despliegue	Nodos, artefactos desplegados y vínculos

MOD-02 Casos de uso en dos granos. [N1][N2]

La plataforma debe generar un diagrama de casos de uso de grano grueso por proyecto, a partir de los requisitos aprobados, y uno detallado por cada componente de la estructura de desglose, a partir de los requisitos que contribuyen a él.

Verificación: Ambos diagramas se regeneran ante cualquier cambio en el conjunto de requisitos o en la contribución declarada.

MOD-03 Cadena de derivación trazable. [N1][N2]

Cada elemento de cada diagrama debe conservar el enlace `derivesFrom` hacia el elemento del modelo que lo origina, hasta llegar al requisito. La cadena completa debe ser navegable en ambos sentidos.

Verificación: Seleccionar un mensaje de un diagrama de secuencia permite alcanzar el escenario, el requisito y el entregable correspondientes sin búsqueda manual.

MOD-04 Formato de intercambio. [N1][N2]

Los diagramas deben exportarse en XMI [6] para los de UML, en BPMN y DMN para los suyos, y en representación textual y vectorial para consulta, conforme a la tabla de formatos de SLCP-DOC-001.

Verificación: Cada diagrama se exporta en al menos un formato abierto de intercambio y en uno de presentación.

MOD-05 Cobertura del modelado. [N1][N2]

La plataforma debe reportar los requisitos aprobados que no aparecen en ningún caso de uso y los elementos de diagrama que no derivan de requisito alguno, como extensión de los indicadores de huérfanos de TRC-11.

Verificación: Ambos indicadores valen cero antes de fijar la línea base de la arquitectura.

4 Momento de definir la arquitectura

4.1 Análisis

La pregunta admite dos respuestas equivocadas y frecuentes. Definir la arquitectura después de tener todos los requisitos convierte el proceso en cascada y retrasa hasta el final las decisiones más caras de revertir. Definirla antes de tener ninguno la deja sin fundamento, porque una arquitectura es una respuesta y sin preguntas no hay respuesta que dar.

La salida está en advertir qué requisitos gobiernan realmente la arquitectura. No son los funcionales: casi ninguna decisión arquitectónica se toma porque el sistema deba registrar una matrícula. Son los atributos de calidad de Int [7] y las restricciones: cuántos usuarios concurrentes, qué disponibilidad, qué tiempo de respuesta, qué normativa de datos, qué plataforma de despliegue. Esos son los requisitos arquitectónicamente significativos, y suelen conocerse muy pronto porque proceden del contexto y no del análisis detallado.

Regla propuesta. La definición de la arquitectura comienza cuando los requisitos de un entregable han sido aprobados y sus atributos de calidad especificados, y debe cerrarse antes de la primera generación de código de ese entregable. No espera al conjunto completo de requisitos del proyecto, y no precede a ninguno.

4.2 Una consecuencia que conviene no pasar por alto

Definir la arquitectura es trabajo, y en este sistema todo trabajo pertenece a la estructura de desglose. La arquitectura no es por tanto una condición previa a la planificación, sino contenido de ella: ocupa entregables, componentes y tareas como cualquier otra labor, y su avance se mide igual. Pretender que sea previa obligaría a hacerla fuera del sistema, con lo que quedaría sin trazabilidad y sin registro, que es justamente lo contrario de lo que el producto persigue.

4.3 Requisitos

ARQ-01 Requisitos arquitectónicamente significativos. [N1][N2]

La plataforma debe permitir marcar qué requisitos y qué atributos de calidad son arquitectónicamente significativos, y debe exigir que exista al menos uno antes de habilitar la definición de la arquitectura.

Verificación: La definición de arquitectura no puede iniciarse sin al menos un requisito significativo declarado.

ARQ-02 Puerta previa a la generación. [N1][N2][N3]

Ningún entregable debe poder generar código sin que exista una descripción de arquitectura vigente que lo cubra, conforme a Int [8].

Verificación: La tentativa de generar código de un entregable sin arquitectura vigente se rechaza indicando la condición incumplida.

ARQ-03 Registro de decisiones. [N1][N2]

Cada decisión de arquitectura debe registrarse con sus requisitos motivadores y sus compromisos, conforme a TRC-08, y debe versionarse como cualquier otro artefacto.

*Verificación: Ninguna decisión arquitectónica figura sin enlace *satisfies* hacia al menos un requisito significativo.*

ARQ-04 Arquitectura como trabajo planificado. [N1][N2]

La definición y el refinamiento de la arquitectura deben modelarse como elementos de la estructura de desglose, con sus recursos y su avance.

Verificación: El avance de la arquitectura se mide con el mismo mecanismo que el del resto del trabajo, sin registro paralelo.

4.4 Vista de despliegue**ARQ-05 Despliegue preliminar y detallado. [N1][N2][N3]**

Debe producirse una vista de despliegue preliminar en el momento de definir la arquitectura, derivada de las restricciones de plataforma ya conocidas, y una vista detallada antes de la primera liberación, derivada de los artefactos efectivamente generados.

Verificación: La vista preliminar existe desde la definición de la arquitectura, y la detallada enumera los artefactos reales de la liberación.

El despliegue admite este tratamiento en dos tiempos porque su parte estable se conoce desde el principio — que el producto sea una aplicación web con persistencia relacional es una restricción, no un hallazgo — mientras que su detalle depende de artefactos que aún no existen. Producir la vista detallada antes de tener esos artefactos obligaría a inventarlos, y producir solo la detallada dejaría sin registro las decisiones de infraestructura que condicionan el diseño desde el primer día.

5 Vocabulario añadido

Concepto	Identificador	Pref.	Naturaleza
Escenario de aceptación	Scenario	SCN	Entidad, oráculo
Caso de uso	UseCase	UCS	Entidad derivada
Componente de software	SoftwareComponent	SWC	Elemento arquitectónico
Vista de arquitectura	ArchitectureView	AVW	Proyección del modelo
Nodo de despliegue	DeploymentNode	DPN	Elemento arquitectónico

6 Asuntos abiertos

Tabla 3: Preguntas abiertas de este documento

ID	Pregunta	Valor por defecto propuesto
Q-31	Se acepta <code>SoftwareComponent</code> para el elemento arquitectónico	Cerrado: sí, conservando <code>COMPONENT</code> como nivel de desglose
Q-32	Es obligatorio el uso de los patrones EARS	No: recomendado y verificado por el análisis, nunca bloqueante
Q-33	Debe la plataforma generar diagramas de secuencia por cada escenario	Sí, bajo demanda; generarlos todos de antemano produce volúmenes inmanejables
Q-34	Quién aprueba la descripción de arquitectura	El facilitador, por ser decisión de construcción y no de necesidad; el propietario la consulta
Q-35	Se admite modelado manual de diagramas	Cerrado: sí se admite la edición, y lo que se edita es el modelo. Ver PRO-01

Referencias

- [1] Alistair Mavin, Philip Wilkinson, Adrian Harwood, and Mark Novak. Easy approach to requirements syntax (EARS). In *Proceedings of the 2009 17th IEEE International Requirements Engineering Conference (RE'09)*, pages 317–322, Atlanta, GA, USA, 2009. IEEE. doi: 10.1109/RE.2009.9.
- [2] Shahbaz Siddeeq et al. Epic-organized vs. requirement-aligned Gherkin: An empirical evaluation of LLM-based acceptance criteria generation, 2026. Aceptado en la International Conference on Software Engineering of Emerging Technologies (SEET 2026). <https://arxiv.org/abs/2607.01980>.
- [3] ISO/IEC 19510:2013 *Information technology — Object Management Group Business Process Model and Notation*. International Organization for Standardization, Geneva, Switzerland, 2013. <https://www.iso.org/standard/62652.html>.
- [4] ISO/IEC 19505-2:2012 *Information technology — Object Management Group Unified Modeling Language (OMG UML) — Part 2: Superstructure*. International Organization for Standardization, Geneva, Switzerland, 2012. <https://www.iso.org/standard/52854.html>.
- [5] *Decision Model and Notation (DMN)*. Object Management Group, Needham, MA, USA, 2026. La versión formal vigente debe verificarse en la página de la especificación, dado que circulan borradores 1.6 y 1.7. <https://www.omg.org/spec/DMN>.
- [6] ISO/IEC 19509:2014 *Information technology — Object Management Group XML Metadata Interchange (XMI)*. International Organization for Standardization, Geneva, Switzerland, 2014. <https://www.iso.org/standard/65183.html>.
- [7] ISO/IEC 25010:2023 *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. International Organization for Standardization, Geneva, Switzerland, 2023. Edición 2, 2023-11; cancela y reemplaza a ISO/IEC 25010:2011 junto con ISO/IEC 25002 e ISO/IEC 25019. <https://www.iso.org/standard/78176.html>.
- [8] ISO/IEC/IEEE 42010:2022 *Software, systems and enterprise — Architecture description*. International Organization for Standardization, Geneva, Switzerland, 2022. <https://www.iso.org/standard/74393.html>.